数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

数据库编程
Database Programming

计算机学院　单纯
sherryshan@bit.edu.cn
2024年6月

内容概览

- 12.1 存储过程
- 12.2 函数

12.1 存储过程（1）

- 存储过程是一组SQL语句和逻辑控制的集合
- 数据库系统需要支持创建、删除和修改存储过程的语法
- 存储过程具有简单、安全、高性能等优点
  - 创建的存储过程保存在数据库系统中，在使用时被调出并且在数据库系统本地进行编译执行，一次编译，多次执行，具有很好的执行效率

12.1 存储过程（2）

- 存储过程具有简单、安全、高性能等优点（续）
  - 数据库系统和应用程序之间通常需要大量的数据交互，而存储过程可以将应用的逻辑“下推”给数据库系统，降低数据的传输量
  - 存储过程还具有过程化的控制语句，可以实现固定的业务逻辑，并且通过存储过程的封装，应用程序只需要访问存储过程即可，从而可以使部分基本表对用户透明，提高了数据系统的安全性

12.1 存储过程（3）

- 创建一个存储过程

```
CREATE [ OR REPLACE ] PROCEDURE 存储过程名(
    [IN|OUT] 参数 1 数据类型,
    [IN|OUT] 参数 1 数据类型,
    ...
)
LANGUAGE lang_name
AS
DECLARE
    变量 1 数据类型,
    变量 2 数据类型,
...
BEGIN
    存储过程的程序体
END;
```

实例

- 例1：统计warehouse表中元组的数量

```
CREATE PROCEDURE warehouse_count()
LANGUAGE SQL
AS
SELECT COUNT( * )
FROM warehouse;
```

实例

- 例2：向new_orders表中插入数据

```
CREATE PROCEDURE new_orders_insert
(
    IN o_id INTEGER,
    IN d_id INTEGER,
    IN w_id INTEGER,
)
LANGUAGE SQL
AS
INSERT INTO new_orders VALUES(o_id, d_id, w_id);
```

实例

– 调用存储过程

```
CALL new_orders_insert(1, 1, 1);
```

– 检查存储过程的效果

```
SELECT * FROM new_orders;
  no_o_id | no_d_id | no_w_id
---------+---------+---------
        1 |       1 |       1
(1 row)
```

12.4 存储过程（4）

- 存储过程的修改
  - SQL中没有提供显示的存储过程修改命令，通常需要通过REPLACE关键字来指定使用当前的存储过程替代之前的同名存储过程

实例

- 例3：将例1中定义的存储过程替换为按照地区分组的数量统计

```
CREATE OR REPLACE PROCEDURE warehouse_count()
LANGUAGE SQL
AS
SELECT w_state, w_city, COUNT( * )
FROM warehouse
GROUP BY w_state, w_city;
```

12.2 函数（1）

- 函数的使用方法和存储过程类似

```
CREATE [ OR REPLACE ] FUNCTION 函数名(
[IN|OUT] 参数 1 数据类型,
[IN|OUT] 参数 1 数据类型,
……
)
RETURNS 数据类型
LANGUAGE lang_name
AS
DECLARE
变量 1 数据类型,
变量 2 数据类型,
……
BEGIN
函数的程序体
END;
```

12.2 函数（2）

- 函数可以应用在SQL语句中，而存储过程必须独立调用
- 函数必须指定返回值

实例

- 例4：向new_orders表中插入数据，并将new_orders中的元组数作为返回值

```
CREATE FUNCTION new_orders_insert_func
(
IN o_id INTEGER,
IN d_id INTEGER,
IN w_id INTEGER
)
RETURNS INTEGER
AS
$ $
DECLARE
count INTEGER;
BEGIN
INSERT INTO new_orders VALUES(o_id, d_id, w_id);
SELECT COUNT( * ) INTO count FROM new_orders;
RETURN count;
END;
$ $ LANGUAGE plpgsql;
```

实例

– 执行获得返回值

```
SELECT new_orders_insert_func(1,1,1);
 new_orders_insert_func
------------------------
                      2
(1 row)
```

参考资料

- 第2章，李国良 周敏奇编著，openGauss数据库核心技术，清华大学出版社，2020